

NTI · AWS SOLUTIONS ARCHITECT · GRADUATION TRACK

Lumina Dental

Deploying a scalable, highly available web application on AWS

EC2 · Auto Scaling · Application Load Balancer · RDS PostgreSQL · CloudFront · WAF

Benha University

Faculty of Engineering in Shoubra

Department of Communication and Electronics

Third academic year

Presented by

Zeyad Bilal Gamal

Training course: NTI — AWS Solutions Architect



Project objective

What problem the deployment had to solve

The application — a Next.js frontend, a FastAPI backend and a PostgreSQL database — originally ran on one single server. The objective was to redeploy it on AWS so that it keeps running when a server fails, grows with demand, and exposes only one controlled entry point to the internet.

BEFORE

One server does everything

- Frontend, backend and database on the same EC2 instance
- The server is a single point of failure
- Growth means manually building a bigger server
- The database and SSH port are exposed to the internet

AFTER

A layered, multi-AZ architecture

- Two or more stateless servers across two Availability Zones
- A load balancer is the only public entry point
- Capacity grows and shrinks automatically with demand
- The database is private, managed, and replicated

2

Why this design matters

Five properties the architecture was built to deliver

1 High availability

Minimum of two instances across two Availability Zones, plus a standby database. Losing one server — or one zone — is not an outage.

2 Scalability

The Auto Scaling Group adds instances from 2 up to 6 when CPU rises, and removes them when it falls. No manual work.

3 Security in depth

Compute and database live in private subnets. Only the load balancer is public, and WAF inspects every request before it arrives.

4 Observability

Seven CloudWatch alarms across the scaling group, load balancer, target group and database, all sending email through SNS.

5 Operability

Instances are disposable: built from one Golden AMI, replaced automatically, and reachable through Session Manager without SSH.

6 Cost awareness

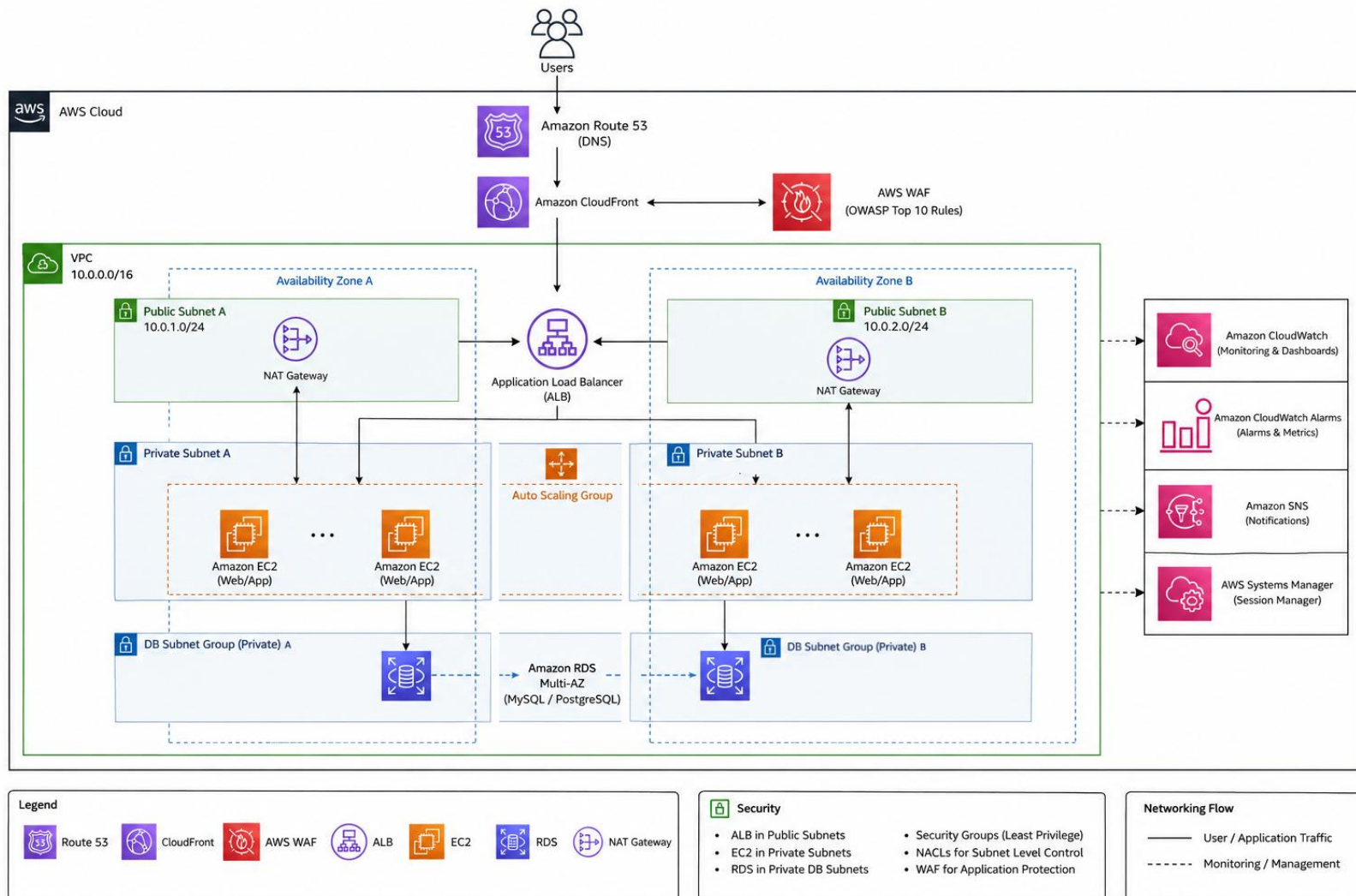
A maximum of six instances puts a ceiling on spend, and every fixed cost in the design buys a specific availability guarantee.

7 Performance

CloudFront caches static assets, reducing latency for users and decreasing the number of requests reaching the origin.

The architecture at a glance

Edge services, a two-zone VPC, and three subnet tiers



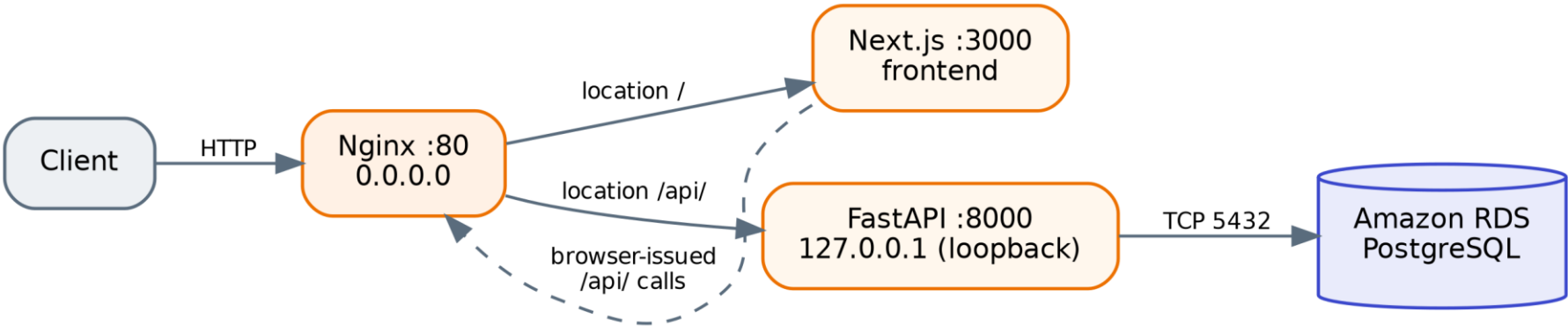
READING THE DIAGRAM

- Users reach CloudFront, where WAF inspects the request
- The Application Load Balancer is the only public entry point
- EC2 instances run in private subnets in two Availability Zones
- RDS sits in isolated database subnets with no internet route
- NAT Gateways give instances outbound access only
- CloudWatch, SNS and Systems Manager sit outside the traffic path

4

The application on each instance

One public port, three internal services



Nginx :80

The only process listening on the network. It receives every request and decides where it goes.

Next.js :3000

Serves the pages. Requests to / are proxied to it internally.

FastAPI :8000

Bound to 127.0.0.1, so it cannot be reached from outside the instance at all. Requests to /api/ are proxied to it.

Amazon EC2 and the Golden AMI

Building one validated server, then cloning it

WHAT I IMPLEMENTED

- Configured one t3.small instance with Nginx, the Next.js build, FastAPI under Uvicorn and the RDS connection
- Verified the backend with curl — it returned `{"status":"ok","service":"Lumina Dental API"}`
- Rebooted the instance and confirmed all three services came back automatically
- Captured that validated instance as the Golden AMI used by every later instance

WHY I CHOSE THIS SERVICE

- EC2 runs a conventional server stack directly, so the application needed no rewriting
- Baking the environment into an AMI means every new instance is identical, with nothing to configure at boot
- The reboot test matters because Auto Scaling launches instances with nobody watching
- The AMI holds the application only — the database stays outside, which is what makes instances disposable

Amazon VPC and the subnet design

The private network everything else lives inside

WHAT I IMPLEMENTED

- Created the VPC aws-project-vpc with the CIDR 10.0.0.0/16
- Divided it into six subnets across two Availability Zones
- Public subnets 10.0.1.0/24 and 10.0.2.0/24 for the NAT Gateways and the load balancer
- Private application subnets 10.0.11.0/24 and 10.0.12.0/24 for the EC2 instances
- Private database subnets 10.0.21.0/24 and 10.0.22.0/24 for RDS
- Five route tables — the database ones carry no default route at all

WHY I CHOSE THIS SERVICE

- The VPC is the boundary that makes every later security control possible
- Three tiers instead of two: the application needs outbound internet access, the database needs none
- Two Availability Zones so that losing one AWS facility does not take the system down
- A subnet is only "private" because of its route table — that is where the isolation actually lives



Internet Gateway and NAT Gateways

Controlling which direction traffic can flow

WHAT I IMPLEMENTED

- Attached the Internet Gateway aws-project-igw and gave the public route table a 0.0.0.0/0 route to it
- Created two NAT Gateways, one in each public subnet, each with its own Elastic IP
- Pointed each private application route table at the NAT Gateway in its own Availability Zone
- Left the database route tables with the local route only — no path to the internet

WHY I CHOSE THIS SERVICE

- Private instances still need outbound access for Systems Manager and package updates
- A NAT Gateway allows outbound connections only; it never accepts unsolicited inbound traffic
- One NAT per zone costs more, but it means a zone failure cannot cut off the healthy zone
- It is a managed service, so there is no NAT instance to patch or keep highly available myself

8

Security Groups

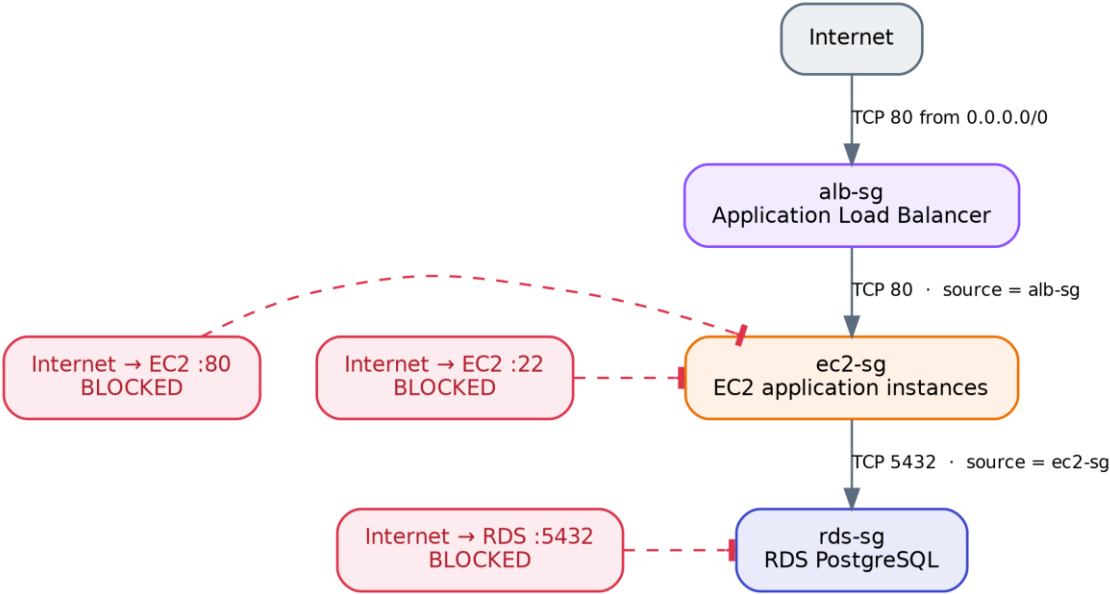
A separate firewall for each tier, chained together

WHAT I IMPLEMENTED

- alb-sg allows TCP 80 from the internet into the load balancer
- ec2-sg allows TCP 80 from alb-sg only — and no SSH rule at all
- rds-sg allows TCP 5432 from ec2-sg only
- Direct internet access to the instances and to the database is therefore blocked

WHY THIS SERVICE

- Each rule references the Security Group above it instead of an IP range
- Instance addresses change constantly under Auto Scaling, so an IP-based rule would be either too broad or unmaintainable
- Authorisation follows group membership, so new instances are trusted automatically



Amazon RDS for PostgreSQL

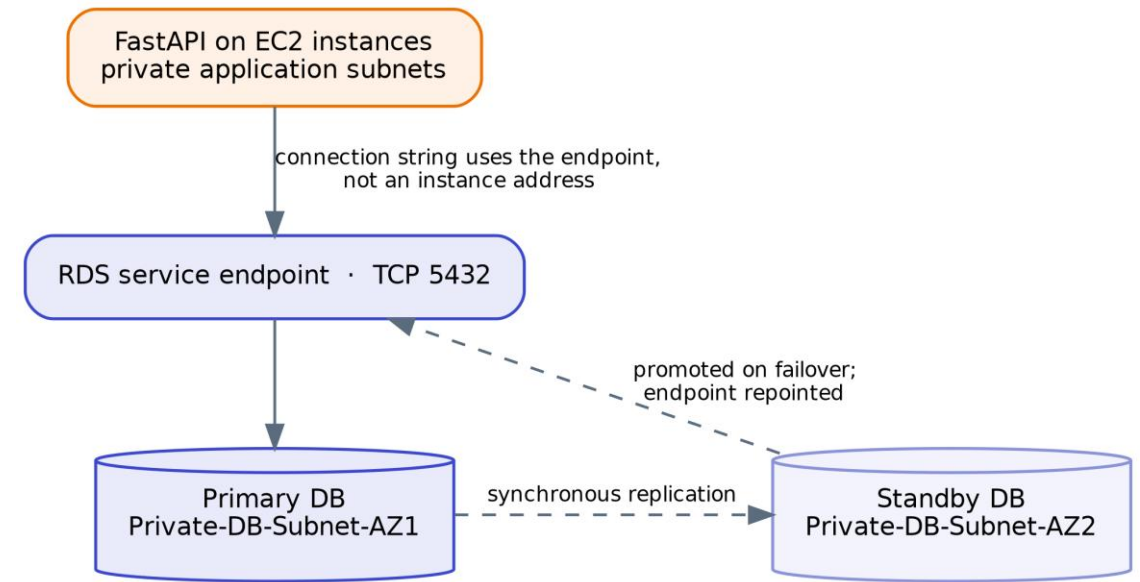
Moving the database off the application servers

WHAT I IMPLEMENTED

- Deployed RDS PostgreSQL into the private database subnets on port 5432
- Configured the application to connect through the RDS endpoint, not an instance address
- Designed it as a Multi-AZ primary with a synchronised standby in the second zone
- Restricted access so that only the application Security Group can connect

WHY THIS SERVICE

- Backups, patching, replication and failover are handled by AWS instead of by me
- Using the endpoint means a failover is transparent to the application
- Keeping state in RDS is what makes the EC2 instances stateless and replaceable



IAM and AWS Systems Manager

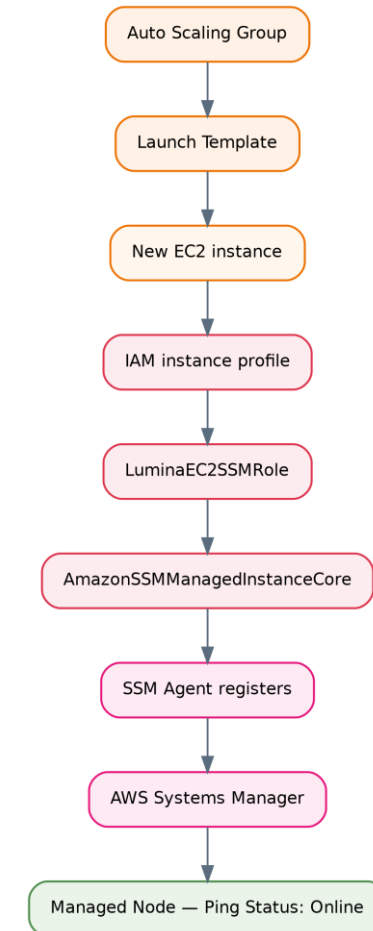
Administrative access without SSH

WHAT I IMPLEMENTED

- Created the IAM role LuminaEC2SSMRole with the managed policy AmazonSSMManagedInstanceCore
- Attached it through the instance profile referenced in the Launch Template
- Verified the SSM Agent on the instance and confirmed it appeared as a Managed Node with status Online
- Opened an interactive Session Manager shell from the console and inspected Nginx and the listening ports

WHY THIS SERVICE

- No SSH port, no key pair to distribute, no public IP and no bastion host to maintain
- Access is controlled by IAM, and only outbound HTTPS is required
- Every instance the Auto Scaling Group launches becomes manageable automatically



EC2 Launch Template

The single definition of how an instance is built

WHAT I IMPLEMENTED

- Created a Launch Template based on the Golden AMI
- Set the instance type to t3.small and attached the application Security Group ec2-sg
- Attached the IAM instance profile carrying the role LuminaEC2SSMRole
- Left subnet placement to the Auto Scaling Group so instances spread across both zones

WHY I CHOSE THIS SERVICE

- Horizontal scaling only means something if instance number two is identical to instance number one
- It is the one place where AMI, instance type, security group and permissions are defined together
- Every instance the Auto Scaling Group creates inherits the same configuration automatically
- Updating the application later is a new template version, not a manual change on each server

EC2 Auto Scaling Group

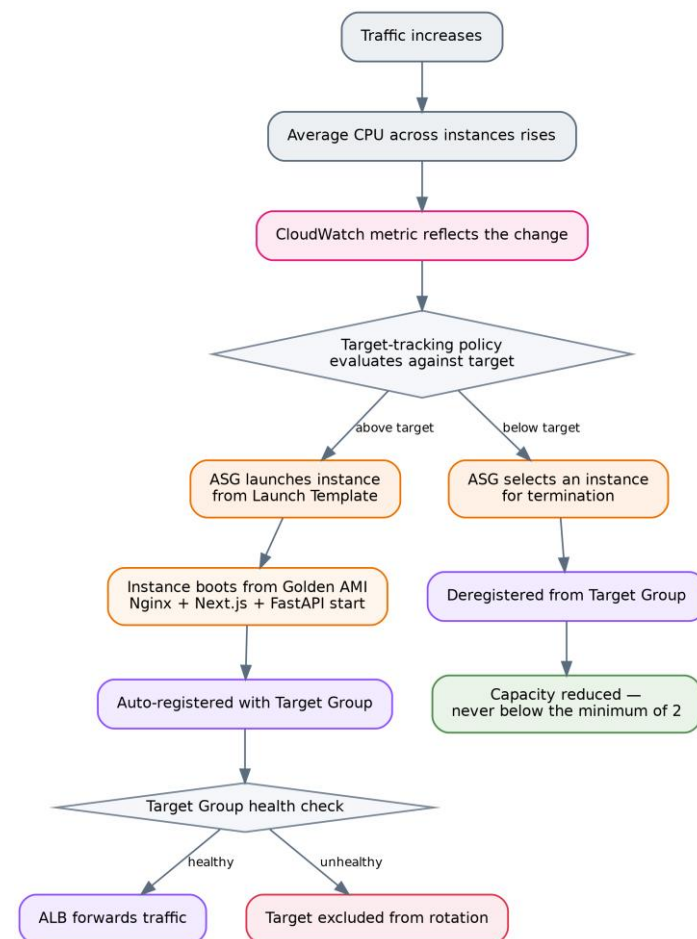
Maintaining capacity and replacing failures automatically

WHAT I IMPLEMENTED

- Created the Auto Scaling Group across both private application subnets
- Set minimum 2, desired 2 and maximum 6 instances
- Attached it to the ALB Target Group so registration is automatic
- Added a target-tracking scaling policy based on CPU utilisation

WHY THIS SERVICE

- A minimum of 2 means a single instance failure is a capacity event, not an outage
- It replaces failed instances by itself — the compute layer becomes self-healing
- The maximum of 6 puts a firm ceiling on both capacity and cost



Application Load Balancer and Target Group

The single public entry point of the application

WHAT I IMPLEMENTED

- Created an internet-facing Application Load Balancer across both Availability Zones
- Configured an HTTP listener on port 80 forwarding to the application Target Group
- Created the Target Group with target type instance, protocol HTTP on port 80, and health checks enabled
- HTTPS was not configured, because a public certificate requires a domain name (see the Route 53 slide)

WHY I CHOSE THIS SERVICE

- It gives the application one stable address that does not depend on any individual instance
- Health checks stop traffic reaching a broken instance within seconds, before any replacement happens
- Because users talk to the ALB, the EC2 instances can stay completely private
- Working at Layer 7 means it understands HTTP, which is what allows path-based health checks and future TLS termination

Amazon CloudFront

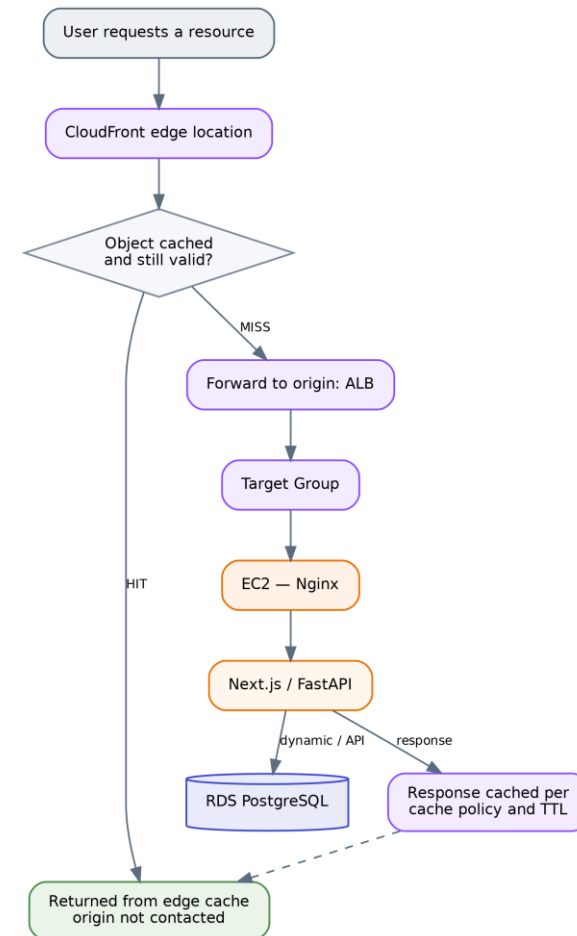
Serving static content from the edge

WHAT I IMPLEMENTED

- Created a distribution with the Application Load Balancer as its origin
- Cached static assets — JavaScript, CSS, images and fonts — at the edge locations
- Left dynamic pages and /api/ requests to pass through to the origin
- CloudFront never talks to the EC2 instances directly; it always goes through the ALB

WHY THIS SERVICE

- Static assets are served closer to the user, which reduces latency
- Cached requests never reach the ALB or the instances, freeing capacity for real work
- API responses depend on the user and the database, so caching them would return stale or wrong data



Amazon CloudWatch and SNS

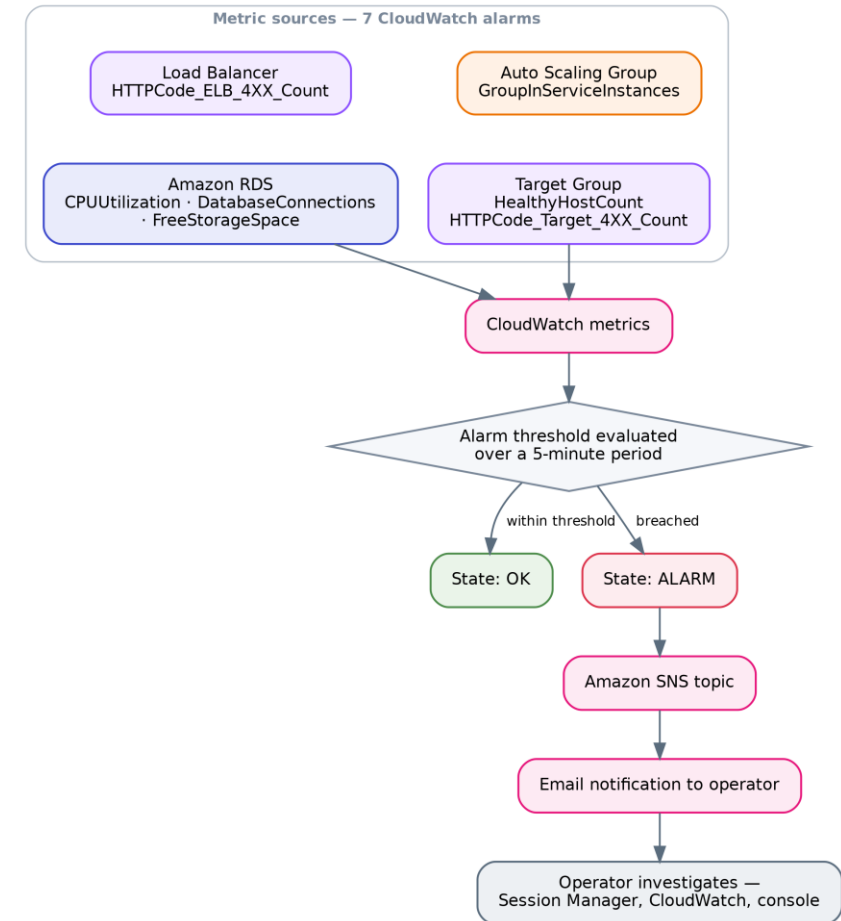
Knowing when something goes wrong

WHAT I IMPLEMENTED

- Configured seven alarms on a five-minute period across four services
- Auto Scaling Group: in-service instances below 2. Target Group: healthy hosts below 2
- Load balancer and targets: more than five 4XX responses in five minutes
- RDS: CPU above 80%, connections above 80, free storage below 2 GB
- Subscribed every alarm to an SNS topic that delivers email notifications

WHY THIS SERVICE

- Alarms are set on group-level metrics, so they survive instances being replaced
- In-service count and healthy host count detect two genuinely different failures
- SNS turns an alarm state change into something a person actually sees



Inspecting requests before they reach the application

WHAT I IMPLEMENTED

- Created a Web ACL combining four AWS managed rule groups: Common, SQL injection, Admin Protection and Anti-DDoS
- Added custom rules: IPv4 and IPv6 allow and block lists, a geographic restriction, a global rate-based rule and a body-size rule
- Tested it: a request to /admin returned HTTP 403, traced through sampled requests to the rule AdminProtection_URI_PATH
- Observed a real internet scanner reaching the public endpoint, confirming WAF inspects live traffic
- Three rules are still in COUNT mode so their behaviour can be measured before blocking

WHY I CHOSE THIS SERVICE

- Security Groups filter by address and port; WAF is the only layer that understands the request itself
- SQL injection protection matters directly here, because the backend talks to a PostgreSQL database
- AWS-managed rule groups stay updated without me writing attack signatures
- COUNT mode lets a rule be validated against real traffic before it can block a legitimate user

Amazon Route 53 and HTTPS

Designed and documented, but deliberately not deployed

Status: planned, not implemented — no domain name was purchased, so no public certificate could be issued.

WHAT THE DESIGN CALLS FOR

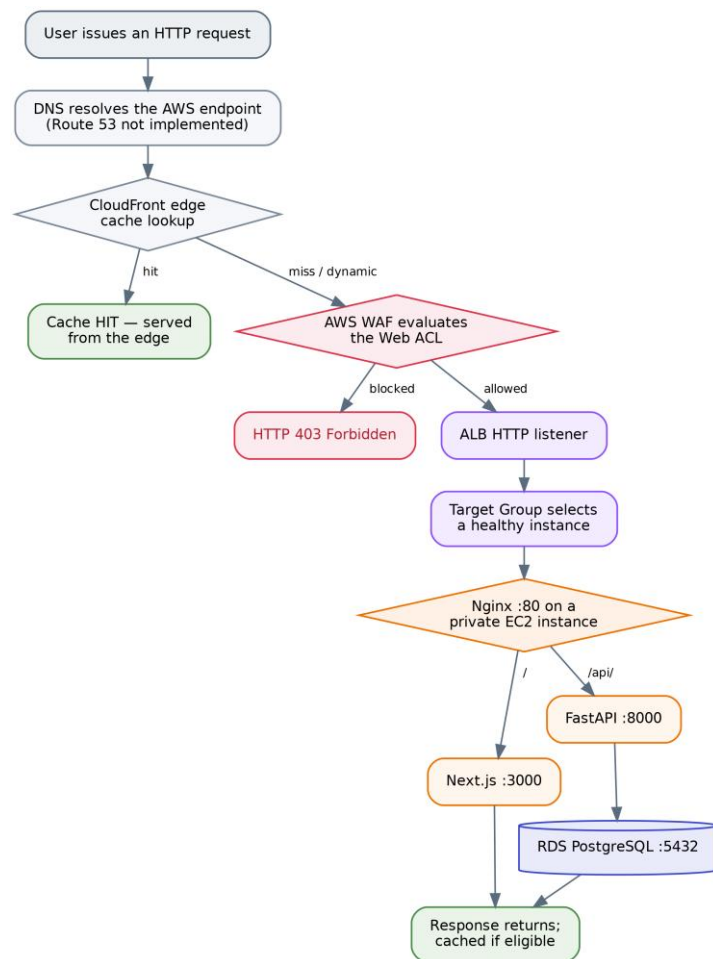
- A Route 53 hosted zone for the purchased domain
- An Alias A record pointing the root domain straight at the Application Load Balancer
- A public ACM certificate validated by DNS
- An HTTPS listener on port 443 with the certificate attached to the ALB
- The port 80 listener changed to redirect to HTTPS

WHY IT BELONGS IN THE DESIGN

- An Alias record can point a root domain at an AWS resource, which a normal CNAME cannot
- Terminating TLS at the load balancer means the certificate is managed in one place, not on every instance
- That matters here because Auto Scaling creates and destroys instances continuously
- The application currently runs over HTTP, which is the most important gap left in the project

End-to-end request flow

How one user request travels through the whole system



THE PATH IN WORDS

- CloudFront answers immediately if the asset is already cached at the edge
- Otherwise WAF inspects the request and can block it with a 403
- The ALB listener forwards the request to the Target Group
- The Target Group picks a healthy private instance
- Nginx sends / to Next.js and /api/ to FastAPI
- FastAPI queries RDS over port 5432 inside the VPC
- The response returns along the same path and may be cached

What happens when something fails

The architecture is judged by how it behaves on a bad day

An instance fails

- Health checks fail and the ALB stops sending it traffic
- The healthy instance keeps serving users
- The Auto Scaling Group launches a replacement from the AMI
- Alarms notify me by email throughout

A whole zone fails

- The load balancer serves from the surviving zone
- Auto Scaling can launch instances there
- That zone has its own NAT Gateway, so outbound access survives
- Capacity is halved, but the service stays up

The database fails

- RDS promotes the standby in the other zone
- The endpoint is repointed to the new primary
- The application reconnects without a configuration change
- There is a short interruption during the failover

Limitations and what I would do next

An honest assessment of the current deployment

CURRENT LIMITATIONS

- Traffic runs over HTTP — no domain, so no TLS certificate
- Database credentials sit in the application configuration rather than in Secrets Manager
- No centralised logging: logs disappear with the instance that produced them
- No alarms on 5XX errors or response time
- The database is a single writable primary — it does not scale horizontally
- Deployment is manual: build an AMI, version the template, replace the instances

PLANNED IMPROVEMENTS

- Buy a domain, add Route 53 and an ACM certificate, and move to HTTPS
- Move credentials into AWS Secrets Manager, read through the existing IAM role
- Send Nginx, application and ALB access logs to CloudWatch Logs and S3
- Add 5XX and latency alarms, and a single CloudWatch dashboard
- Add RDS Proxy for connection pooling and ElastiCache for repeated reads
- Define the infrastructure as code and automate the release with a pipeline

In summary

A single-server application was rebuilt as a layered AWS architecture: stateless compute behind a load balancer, a managed database in isolated subnets, security enforced at every tier, and monitoring that reports failures before users do.

12

AWS services used

2

Availability Zones

2 – 6

instances, automatically

7

CloudWatch alarms

Thank you — questions are welcome

Zeyad Bilal Gamal · Benha University, Faculty of Engineering in Shoubra